Fork-Join Blocks in Verilog HDL

Digital Design and RTL Fundamentals

October 12, 2026

Contents

1 Introduction **2**

2 Sequential Execution Using begin-end **2**
2.1 Example 2

3 Fork-Join Block **3**
3.1 Syntax 3
3.2 Example 3

4 Fork-Join with Different Delays **4**
4.1 Example 4

5 Fork-Join vs. Begin-End **5**

6 The fork-join_any Construct **5**
6.1 Syntax 5
6.2 Example 5

7 The fork-join_none Construct **7**
7.1 Syntax 7
7.2 Example 7

8 The disable fork Statement **8**
8.1 Example 8

9 Named Fork Blocks **8**
9.1 Example 8

10 Practical Example: Parallel Testbench **9**
10.1 Example 9

11 Comparison of Parallel Constructs **10**

12 Common Errors and Best Practices **10**

13 Summary **11**

1 Introduction

Procedural blocks in Verilog HDL normally execute statements sequentially within a `begin-end` block.

Sometimes, multiple procedural statements or processes need to execute concurrently. Verilog provides the `fork-join` construct for this purpose.

A fork-join block starts multiple statements concurrently and waits for all of them to complete before continuing execution.

Fork-join constructs are primarily useful in simulation testbenches, stimulus generation, and verification environments.

The main parallel-block constructs are:

- `fork-join`
- `fork-join_any`
- `fork-join_none`

Important: `join_any` and `join_none` are SystemVerilog constructs. Classic Verilog supports `fork-join`.

2 Sequential Execution Using begin-end

A `begin-end` block groups statements that execute sequentially in the order specified.

2.1 Example

```
module begin_end_example;

    initial begin
        $display("Statement 1");
        #5;
        $display("Statement 2");
        #5;
        $display("Statement 3");
    end

endmodule
```

Expected output:

```
Statement 1
Statement 2
Statement 3
```

The statements execute one after another. The delay statements suspend execution of the current procedural process.

3 Fork-Join Block

The `fork-join` construct starts its child statements concurrently.

The parent process resumes after every child statement has completed.

3.1 Syntax

```
fork
    statement_1;
    statement_2;
    statement_3;
join
```

The statements between `fork` and `join` are independent concurrent procedural statements.

3.2 Example

```
module fork_join_example;
    initial begin
        $display("Start at %0t", $time);
        fork
            begin
                #5;
                $display("Process 1 at %0t", $time);
            end

            begin
                #10;
                $display("Process 2 at %0t", $time);
            end

            begin
                #15;
                $display("Process 3 at %0t", $time);
            end
        join

        $display("All processes completed at %0t",
                 $time);
    end
endmodule
```

Expected output:

```
Start at 0
Process 1 at 5
Process 2 at 10
Process 3 at 15
All processes completed at 15
```

Explanation:

- All three child processes start at simulation time 0.
- Process 1 completes after 5 time units.
- Process 2 completes after 10 time units.
- Process 3 completes after 15 time units.
- The parent process resumes when the last child process completes.

The total elapsed time is 15 time units, not 30 time units, because the child processes execute concurrently.

4 Fork-Join with Different Delays

The completion time of a fork-join block is determined by the longest-running child process, assuming all children terminate normally.

4.1 Example

```
module fork_delay_example;

    initial begin
        $display("Start time = %0t", $time);

        fork
            #4  $display("Task A at %0t", $time);
            #8  $display("Task B at %0t", $time);
            #12 $display("Task C at %0t", $time);
        join

        $display("End time = %0t", $time);
        $finish;
    end

endmodule
```

Expected output:

```
Start time = 0
Task A at 4
Task B at 8
Task C at 12
End time = 12
```

All child statements begin concurrently. The parent resumes at time 12 because the longest child statement finishes at that time.

5 Fork-Join vs. Begin-End

Feature	begin-end	fork-join
Execution	Sequential	Concurrent child processes
Starting time	Statements execute in order	Child processes start together
Completion	After the final statement	After all children complete
Timing	Delays accumulate sequentially	Processes overlap in time
Typical use	Procedural algorithms	Parallel testbench activities

6 The fork-join_any Construct

The `fork-join_any` construct starts multiple child processes and allows the parent to continue as soon as any one child completes.

It is available in SystemVerilog, not classic Verilog.

6.1 Syntax

```
fork
    statement_1;
    statement_2;
    statement_3;
join_any
```

The parent process does not wait for all child processes to finish.

The other child processes may continue running after the parent resumes.

6.2 Example

```
module fork_join_any_example;

    initial begin
        $display("Start at %0t", $time);

        fork
            begin
                #5;
                $display("Process 1 completed at %0t",
                         $time);
            end

            begin
                #10;
                $display("Process 2 completed at %0t",
                         $time);
            end

            begin
```

```
                #15;
                $display("Process 3 completed at %0t",
                        $time);
            end
        join_any

        $display("Parent resumed at %0t", $time);

        #20;
        $finish;
    end

endmodule
```

Expected output before simulation ends:

```
Start at 0
Process 1 completed at 5
Parent resumed at 5
Process 2 completed at 10
Process 3 completed at 15
```

The parent resumes when Process 1 completes at time 5. Processes 2 and 3 continue running because they have not been disabled.

Note: If multiple child processes complete at the same simulation time, event scheduling determines the ordering of messages.

7 The fork-join_none Construct

The `fork-join_none` construct starts child processes without making the parent wait for their completion.

It is a SystemVerilog construct.

7.1 Syntax

```
fork
    statement_1;
    statement_2;
join_none
```

The parent continues immediately after spawning the child processes. The spawned processes begin executing when the parent process subsequently suspends or terminates, subject to the language scheduling rules.

7.2 Example

```
module fork_join_none_example;

    initial begin
        $display("Parent starts at %0t", $time);

        fork
            begin
                #5;
                $display("Child 1 at %0t", $time);
            end

            begin
                #10;
                $display("Child 2 at %0t", $time);
            end
        join_none

        $display("Parent continues at %0t", $time);

        #15;
        $display("Parent finishes at %0t", $time);

        $finish;
    end

endmodule
```

The parent does not wait at the `join_none` statement. It continues with its next statement.

The child processes execute independently and can finish while the parent is waiting at a later delay.

8 The disable fork Statement

In SystemVerilog, `disable fork` terminates active child processes associated with the calling process according to the fork-process hierarchy rules.

It is commonly used with `join_any` to stop remaining child processes after one of them completes.

8.1 Example

```
module disable_fork_example;

    initial begin
        $display("Start at %0t", $time);

        fork
            begin
                #5;
                $display("Fast process completed");
            end

            begin
                #20;
                $display("Slow process completed");
            end
        join_any

        disable fork;

        $display("Remaining child processes stopped");
        $finish;
    end

endmodule
```

The first child completes after 5 time units. The parent resumes, and `disable fork` terminates the remaining active child process.

The message from the slow process is not printed because that process is terminated.

Note: This example uses SystemVerilog. Use `disable fork` carefully when nested or unrelated child processes exist.

9 Named Fork Blocks

A fork block can be named using a named block. Named blocks make it possible to refer to specific procedural blocks when using `disable`.

9.1 Example

```
module named_fork_example;

    initial begin
        fork : worker_group
            begin
                #5;
                $display("Worker 1 completed");
            end

            begin
                #10;
                $display("Worker 2 completed");
            end
        join
    end

endmodule
```

The name `worker_group` identifies the named fork block.

Named blocks are useful for organizing procedural code and controlling execution when appropriate.

10 Practical Example: Parallel Testbench

A testbench may need to apply stimulus while another process monitors the simulation.

10.1 Example

```
`timescale 1ns/1ps

module parallel_testbench;

    reg clk;
    reg reset;

    initial begin
        clk = 0;

        forever #5 clk = ~clk;
    end

    initial begin
        reset = 1;

        #12 reset = 0;

        #30 $finish;
    end

```

```
22      initial begin
23          $monitor("Time=%0t clk=%b reset=%b",
24                   $time, clk, reset);
25      end
26
27  endmodule
```

The three initial processes operate concurrently:

- The first process generates a clock.
- The second process controls reset and terminates simulation.
- The third process reports signal changes.

This illustrates concurrent procedural execution at the module level. Multiple initial blocks are independent processes even without an explicit fork-join construct.

11 Comparison of Parallel Constructs

Construct	Language	Parent resumes
`fork-join`	Verilog and SystemVerilog	After all child processes complete
`fork-join_any`	SystemVerilog	After at least one child completes
`fork-join_none`	SystemVerilog	Without waiting for child completion

12 Common Errors and Best Practices

Common errors include:

- Expecting fork-join child processes to execute sequentially.
- Assuming `join_any` terminates all unfinished child processes automatically.
- Assuming `join_none` waits for child processes to complete.
- Forgetting that concurrent processes may access the same variables.
- Using SystemVerilog constructs while compiling in a Verilog-only language mode.
- Using `disable fork` without considering the active process hierarchy.

Recommended practices include:

- Use `begin-end` for sequential procedural operations.
- Use `fork-join` when all child processes must finish before continuing.
- Use `fork-join_any` when the first completed child should release the parent.
- Use `fork-join_none` for spawning concurrent testbench activities.
- Use suitable synchronization when multiple processes share data.
- Keep parallel stimulus and monitoring processes organized and easy to debug.

13 Summary

- `begin-end` groups sequential procedural statements.
- `fork-join` executes child processes concurrently and waits for all of them to complete.
- `fork-join_any` waits for at least one child to complete.
- `fork-join_none` does not wait for child processes to complete.
- `disable fork` can terminate active child processes in SystemVerilog.
- Fork-join constructs are particularly useful for simulation testbenches and verification environments.

Key takeaway: Fork-join constructs coordinate concurrent procedural activities. Choose the appropriate join behavior based on whether the parent must wait for all children, any child, or none of them.